JavaScript is disabled on your browser.



compact1, compact2, compact3

java.util

# Class Spliterators

java.lang.Object
    java.util.Spliterators

---

```
public final class Spliterators
extends Object
```

Static classes and methods for operating on or creating instances of `Spliterator` and its primitive specializations `Spliterator.OfInt`, `Spliterator.OfLong`, and `Spliterator.OfDouble`.

**Since:**

1.8

**See Also:**

`Spliterator`

## *Nested Class Summary*

**Nested Classes**

| Modifier and Type | Class | Description |
|---|---|---|
| static class | **Spliterators.AbstractDoubleSpliterator** | An abstract `Spliterator.OfDouble` that implements `trySplit` to permit limited parallelism. |
| static class | **Spliterators.AbstractIntSpliterator** | An abstract `Spliterator.OfInt` that implements |

| | | |
|---|---|---|
| | | trySplit to permit limited parallelism. |
| static class | Spliterators.AbstractLongSpliterator | An abstract Spliterator.OfLong that implements trySplit to permit limited parallelism. |
| static class | Spliterators.AbstractSpliterator<T> | An abstract Spliterator that implements trySplit to permit limited parallelism. |

## *Method Summary*

Concrete Methods

| Modifier and Type | Method |
|---|---|
| static Spliterator.OfDouble | emptyDoubleSpliterator() |
| static Spliterator.OfInt | emptyIntSpliterator() |
| static Spliterator.OfLong | emptyLongSpliterator() |
| static <T> Spliterator<T> | emptySpliterator() |
| static PrimitiveIterator.OfDouble | iterator(Spliterator.OfDouble spliterator) |
| static PrimitiveIterator.OfInt | iterator(Spliterator.OfInt spliterator) |
| static PrimitiveIterator.OfLong | iterator(Spliterator.OfLong spliterator) |
| static <T> Iterator<T> | iterator(Spliterator<? extends T> spliterator) |
| static <T> Spliterator<T> | spliterator(Collection<? extends T> c, int cha |

| | |
|---|---|
| static **Spliterator.OfDouble** | **spliterator**(double[] array, int additionalChar |
| static **Spliterator.OfDouble** | **spliterator**(double[] array, int fromIndex, int<br>int additionalCharacteristics) |
| static **Spliterator.OfInt** | **spliterator**(int[] array, int additionalCharact |
| static **Spliterator.OfInt** | **spliterator**(int[] array, int fromIndex, int to<br>int additionalCharacteristics) |
| static <T> **Spliterator**<T> | **spliterator**(**Iterator**<? extends T> iterator, lo<br>int characteristics) |
| static **Spliterator.OfLong** | **spliterator**(long[] array, int additionalCharac |
| static **Spliterator.OfLong** | **spliterator**(long[] array, int fromIndex, int t<br>int additionalCharacteristics) |
| static <T> **Spliterator**<T> | **spliterator**(**Object**[] array, int additionalChar |

| | |
|---|---|
| static <T> **Spliterator**<T> | **spliterator**(**Object**[] array, int fromIndex, int<br>int additionalCharacteristics) |
| static **Spliterator.OfDouble** | **spliterator**(**PrimitiveIterator.OfDouble** iterato<br>int characteristics) |
| static **Spliterator.OfInt** | **spliterator**(**PrimitiveIterator.OfInt** iterator,<br>int characteristics) |
| static **Spliterator.OfLong** | **spliterator**(**PrimitiveIterator.OfLong** iterator,<br>int characteristics) |
| static <T> **Spliterator**<T> | **spliteratorUnknownSize**(**Iterator**<? extends T> i<br>int characteristics) |
| static **Spliterator.OfDouble** | **spliteratorUnknownSize**(**PrimitiveIterator.OfDou**<br>int characteristics) |
| static **Spliterator.OfInt** | **spliteratorUnknownSize**(**PrimitiveIterator.OfInt**<br>int characteristics) |

| | |
|---|---|
| static Spliterator.OfLong | spliteratorUnknownSize(PrimitiveIterator.OfLon<br>int characteristics) |

### Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

## *Method Detail*

### emptySpliterator

```
public static <T> Spliterator<T> emptySpliterator()
```

Creates an empty Spliterator

The empty spliterator reports Spliterator.SIZED and Spliterator.SUBSIZED. Calls to Spliterator.trySplit() always return null.

**Type Parameters:**

T - Type of elements

**Returns:**

An empty spliterator

### emptyIntSpliterator

```
public static Spliterator.OfInt emptyIntSpliterator()
```

Creates an empty Spliterator.OfInt

The empty spliterator reports Spliterator.SIZED and Spliterator.SUBSIZED. Calls to Spliterator.trySplit() always return null.

**Returns:**

An empty spliterator

### emptyLongSpliterator

```
public static Spliterator.OfLong emptyLongSpliterator()
```

Creates an empty `Spliterator.OfLong`

The empty spliterator reports `Spliterator.SIZED` and `Spliterator.SUBSIZED`. Calls to `Spliterator.trySplit()` always return `null`.

**Returns:**

`An empty spliterator`

### emptyDoubleSpliterator

```
public static Spliterator.OfDouble emptyDoubleSpliterator()
```

Creates an empty `Spliterator.OfDouble`

The empty spliterator reports `Spliterator.SIZED` and `Spliterator.SUBSIZED`. Calls to `Spliterator.trySplit()` always return `null`.

**Returns:**

`An empty spliterator`

### spliterator

```
public static <T> Spliterator<T> spliterator(Object[] array,
                                             int additionalCharacteristics)
```

Creates a `Spliterator` covering the elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use `Arrays.spliterator(Object[])`.

The returned spliterator always reports the characteristics `SIZED` and `SUBSIZED`. The caller may provide additional characteristics for the spliterator to report; it is common to additionally specify `IMMUTABLE` and `ORDERED`.

**Type Parameters:**

`T - Type of elements`

**Parameters:**

array - The array, assumed to be unmodified during use

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

**See Also:**

Arrays.spliterator(Object[])

### spliterator

```
public static <T> Spliterator<T> spliterator(Object[] array,
                                             int fromIndex,
                                             int toIndex,
                                             int additionalCharacteristics)
```

Creates a Spliterator covering a range of elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use Arrays.spliterator(Object[]).

The returned spliterator always reports the characteristics SIZED and SUBSIZED. The caller may provide additional characteristics for the spliterator to report; it is common to additionally specify IMMUTABLE and ORDERED.

**Type Parameters:**

T - Type of elements

**Parameters:**

array - The array, assumed to be unmodified during use

fromIndex - The least index (inclusive) to cover

toIndex - One past the greatest index to cover

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

ArrayIndexOutOfBoundsException - if fromIndex is negative, toIndex is less than fromIndex, or toIndex is greater than the array size

**See Also:**

Arrays.spliterator(Object[], int, int)

### spliterator

```
public static Spliterator.OfInt spliterator(int[] array,
                                            int additionalCharacteristics)
```

Creates a Spliterator.OfInt covering the elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use Arrays.spliterator(int[]).

The returned spliterator always reports the characteristics SIZED and SUBSIZED. The caller may provide additional characteristics for the spliterator to report; it is common to additionally specify IMMUTABLE and ORDERED.

**Parameters:**

array - The array, assumed to be unmodified during use

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

**See Also:**

Arrays.spliterator(int[])

### spliterator

```
public static Spliterator.OfInt spliterator(int[] array,
                                            int fromIndex,
                                            int toIndex,
                                            int additionalCharacteristics)
```

Creates a `Spliterator.OfInt` covering a range of elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use `Arrays.spliterator(int[], int, int)`.

The returned spliterator always reports the characteristics `SIZED` and `SUBSIZED`. The caller may provide additional characteristics for the spliterator to report; it is common to additionally specify `IMMUTABLE` and `ORDERED`.

**Parameters:**

`array - The array, assumed to be unmodified during use`

`fromIndex - The least index (inclusive) to cover`

`toIndex - One past the greatest index to cover`

`additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported`

**Returns:**

`A spliterator for an array`

**Throws:**

`NullPointerException - if the given array is null`

`ArrayIndexOutOfBoundsException - if fromIndex is negative, toIndex is less than fromIndex, or toIndex is greater than the array size`

**See Also:**

`Arrays.spliterator(int[], int, int)`

### spliterator

```
public static Spliterator.OfLong spliterator(long[] array,
                                             int additionalCharacteristics)
```

Creates a `Spliterator.OfLong` covering the elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use `Arrays.spliterator(long[])`.

The returned spliterator always reports the characteristics `SIZED` and `SUBSIZED`. The caller may provide additional characteristics for the

spliterator to report; it is common to additionally specify IMMUTABLE and ORDERED.

**Parameters:**

array - The array, assumed to be unmodified during use

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

**See Also:**

Arrays.spliterator(long[])

### spliterator

```
public static Spliterator.OfLong spliterator(long[] array,
                                             int fromIndex,
                                             int toIndex,
                                             int additionalCharacteristics)
```

Creates a Spliterator.OfLong covering a range of elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use Arrays.spliterator(long[], int, int).

The returned spliterator always reports the characteristics SIZED and SUBSIZED. The caller may provide additional characteristics for the spliterator to report. (For example, if it is known the array will not be further modified, specify IMMUTABLE; if the array data is considered to have an an encounter order, specify ORDERED). The method Arrays.spliterator(long[], int, int) can often be used instead, which returns a spliterator that reports SIZED, SUBSIZED, IMMUTABLE, and ORDERED.

**Parameters:**

array - The array, assumed to be unmodified during use

fromIndex - The least index (inclusive) to cover

toIndex - One past the greatest index to cover

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

ArrayIndexOutOfBoundsException - if fromIndex is negative, toIndex is less than fromIndex, or toIndex is greater than the array size

**See Also:**

Arrays.spliterator(long[], int, int)

### spliterator

```
public static Spliterator.OfDouble spliterator(double[] array,
                                               int additionalCharacteristics)
```

Creates a Spliterator.OfDouble covering the elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use Arrays.spliterator(double[]).

The returned spliterator always reports the characteristics SIZED and SUBSIZED. The caller may provide additional characteristics for the spliterator to report; it is common to additionally specify IMMUTABLE and ORDERED.

**Parameters:**

array - The array, assumed to be unmodified during use

additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported

**Returns:**

A spliterator for an array

**Throws:**

NullPointerException - if the given array is null

**See Also:**

Arrays.spliterator(double[])

### spliterator

```
public static Spliterator.OfDouble spliterator(double[] array,
                                               int fromIndex,
```

```
                                                int toIndex,
                                                int additionalCharacteristics)
```

Creates a `Spliterator.OfDouble` covering a range of elements of a given array, using a customized set of spliterator characteristics.

This method is provided as an implementation convenience for Spliterators which store portions of their elements in arrays, and need fine control over Spliterator characteristics. Most other situations in which a Spliterator for an array is needed should use `Arrays.spliterator(double[], int, int)`.

The returned spliterator always reports the characteristics `SIZED` and `SUBSIZED`. The caller may provide additional characteristics for the spliterator to report. (For example, if it is known the array will not be further modified, specify `IMMUTABLE`; if the array data is considered to have an an encounter order, specify `ORDERED`). The method `Arrays.spliterator(long[], int, int)` can often be used instead, which returns a spliterator that reports `SIZED`, `SUBSIZED`, `IMMUTABLE`, and `ORDERED`.

**Parameters:**

`array - The array, assumed to be unmodified during use`

`fromIndex - The least index (inclusive) to cover`

`toIndex - One past the greatest index to cover`

`additionalCharacteristics - Additional spliterator characteristics of this spliterator's source or elements beyond SIZED and SUBSIZED which are are always reported`

**Returns:**

`A spliterator for an array`

**Throws:**

`NullPointerException - if the given array is null`

`ArrayIndexOutOfBoundsException - if fromIndex is negative, toIndex is less than fromIndex, or toIndex is greater than the array size`

**See Also:**

`Arrays.spliterator(double[], int, int)`

### spliterator

```
public static <T> Spliterator<T> spliterator(Collection<? extends T> c,
                                             int characteristics)
```

Creates a `Spliterator` using the given collection's `Collection.iterator()` as the source of elements, and reporting its `Collection.size()` as its initial size.

The spliterator is *late-binding*, inherits the *fail-fast* properties of the collection's iterator, and implements `trySplit` to permit limited parallelism.

**Type Parameters:**

`T - Type of elements`

**Parameters:**

`c - The collection`

`characteristics - Characteristics of this spliterator's source or elements. The characteristics SIZED and SUBSIZED are additionally reported unless CONCURRENT is supplied.`

**Returns:**

`A spliterator from an iterator`

**Throws:**

`NullPointerException - if the given collection is null`

### spliterator

```
public static <T> Spliterator<T> spliterator(Iterator<? extends T> iterator,
                                             long size,
                                             int characteristics)
```

Creates a `Spliterator` using a given `Iterator` as the source of elements, and with a given initially reported size.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements `trySplit` to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned, or the initially reported size is not equal to the actual number of elements in the source.

**Type Parameters:**

`T - Type of elements`

**Parameters:**

`iterator - The iterator for the source`

`size - The number of elements in the source, to be reported as initial estimateSize`

`characteristics - Characteristics of this spliterator's source or elements. The characteristics SIZED and SUBSIZED are additionally reported unless CONCURRENT is supplied.`

**Returns:**

`A spliterator from an iterator`

**Throws:**

NullPointerException - if the given iterator is null

### spliteratorUnknownSize

```
public static <T> Spliterator<T> spliteratorUnknownSize(Iterator<? extends T> ite
                                                        int characteristics)
```

Creates a Spliterator using a given Iterator as the source of elements, with no initial size estimate.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements trySplit to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned.

**Type Parameters:**

T - Type of elements

**Parameters:**

iterator - The iterator for the source

characteristics - Characteristics of this spliterator's source or elements (SIZED and SUBSIZED, if supplied, are ignored and are not reported.)

**Returns:**

A spliterator from an iterator

**Throws:**

NullPointerException - if the given iterator is null

### spliterator

```
public static Spliterator.OfInt spliterator(PrimitiveIterator.OfInt iterator,
                                            long size,
                                            int characteristics)
```

Creates a Spliterator.OfInt using a given IntStream.IntIterator as the source of elements, and with a given initially reported size.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements trySplit to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned, or the initially reported size is not equal to the actual number of elements in the source.

**Parameters:**

`iterator - The iterator for the source`

`size - The number of elements in the source, to be reported as initial estimateSize.`

`characteristics - Characteristics of this spliterator's source or elements. The characteristics SIZED and SUBSIZED are additionally reported unless CONCURRENT is supplied.`

**Returns:**

`A spliterator from an iterator`

**Throws:**

`NullPointerException - if the given iterator is null`

### spliteratorUnknownSize

```
public static Spliterator.OfInt spliteratorUnknownSize(PrimitiveIterator.OfInt it
                                                       int characteristics)
```

Creates a `Spliterator.OfInt` using a given `IntStream.IntIterator` as the source of elements, with no initial size estimate.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements `trySplit` to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned.

**Parameters:**

`iterator - The iterator for the source`

`characteristics - Characteristics of this spliterator's source or elements (SIZED and SUBSIZED, if supplied, are ignored and are not reported.)`

**Returns:**

`A spliterator from an iterator`

**Throws:**

`NullPointerException - if the given iterator is null`

### spliterator

```
public static Spliterator.OfLong spliterator(PrimitiveIterator.OfLong iterator,
                                             long size,
                                             int characteristics)
```

Creates a `Spliterator.OfLong` using a given `LongStream.LongIterator` as the source of elements, and with a given initially reported size.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements `trySplit` to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned, or the initially reported size is not equal to the actual number of elements in the source.

**Parameters:**

`iterator - The iterator for the source`

`size - The number of elements in the source, to be reported as initial estimateSize.`

`characteristics - Characteristics of this spliterator's source or elements. The characteristics SIZED and SUBSIZED are additionally reported unless CONCURRENT is supplied.`

**Returns:**

`A spliterator from an iterator`

**Throws:**

`NullPointerException - if the given iterator is null`

### spliteratorUnknownSize

```
public static Spliterator.OfLong spliteratorUnknownSize(PrimitiveIterator.OfLong
                                                         int characteristics)
```

Creates a `Spliterator.OfLong` using a given `LongStream.LongIterator` as the source of elements, with no initial size estimate.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements `trySplit` to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned.

**Parameters:**

`iterator - The iterator for the source`

`characteristics - Characteristics of this spliterator's source or elements (SIZED and SUBSIZED, if supplied, are ignored and are not reported.)`

**Returns:**

`A spliterator from an iterator`

**Throws:**

NullPointerException - if the given iterator is null

### spliterator

```
public static Spliterator.OfDouble spliterator(PrimitiveIterator.OfDouble iterato
                                               long size,
                                               int characteristics)
```

Creates a Spliterator.OfDouble using a given DoubleStream.DoubleIterator as the source of elements, and with a given initially reported size.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements trySplit to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned, or the initially reported size is not equal to the actual number of elements in the source.

**Parameters:**

iterator - The iterator for the source

size - The number of elements in the source, to be reported as initial estimateSize

characteristics - Characteristics of this spliterator's source or elements. The characteristics SIZED and SUBSIZED are additionally reported unless CONCURRENT is supplied.

**Returns:**

A spliterator from an iterator

**Throws:**

NullPointerException - if the given iterator is null

### spliteratorUnknownSize

```
public static Spliterator.OfDouble spliteratorUnknownSize(PrimitiveIterator.OfDou
                                                          int characteristics)
```

Creates a Spliterator.OfDouble using a given DoubleStream.DoubleIterator as the source of elements, with no initial size estimate.

The spliterator is not *late-binding*, inherits the *fail-fast* properties of the iterator, and implements trySplit to permit limited parallelism.

Traversal of elements should be accomplished through the spliterator. The behaviour of splitting and traversal is undefined if the iterator is operated on after the spliterator is returned.

**Parameters:**

`iterator - The iterator for the source`

`characteristics - Characteristics of this spliterator's source or elements (SIZED and SUBSIZED, if supplied, are ignored and are not reported.)`

**Returns:**

`A spliterator from an iterator`

**Throws:**

`NullPointerException - if the given iterator is null`

### iterator

```
public static <T> Iterator<T> iterator(Spliterator<? extends T> spliterator)
```

Creates an `Iterator` from a `Spliterator`.

Traversal of elements should be accomplished through the iterator. The behaviour of traversal is undefined if the spliterator is operated after the iterator is returned.

**Type Parameters:**

`T - Type of elements`

**Parameters:**

`spliterator - The spliterator`

**Returns:**

`An iterator`

**Throws:**

`NullPointerException - if the given spliterator is null`

### iterator

```
public static PrimitiveIterator.OfInt iterator(Spliterator.OfInt spliterator)
```

Creates an `PrimitiveIterator.OfInt` from a `Spliterator.OfInt`.

Traversal of elements should be accomplished through the iterator. The behaviour of traversal is undefined if the spliterator is operated after the iterator is returned.

**Parameters:**

spliterator - The spliterator

**Returns:**

An iterator

**Throws:**

NullPointerException - if the given spliterator is null

### iterator

public static PrimitiveIterator.OfLong iterator(Spliterator.OfLong spliterator)

Creates an PrimitiveIterator.OfLong from a Spliterator.OfLong.

Traversal of elements should be accomplished through the iterator. The behaviour of traversal is undefined if the spliterator is operated after the iterator is returned.

**Parameters:**

spliterator - The spliterator

**Returns:**

An iterator

**Throws:**

NullPointerException - if the given spliterator is null

### iterator

public static PrimitiveIterator.OfDouble iterator(Spliterator.OfDouble spliterato

Creates an PrimitiveIterator.OfDouble from a Spliterator.OfDouble.

Traversal of elements should be accomplished through the iterator. The behaviour of traversal is undefined if the spliterator is operated after the iterator is returned.

**Parameters:**

spliterator - The spliterator

**Returns:**

An iterator

**Throws:**

C

```
NullPointerException - if the given spliterator is null
```




Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.